

Arquitectura del Sistema – Sistema de Inventario y Ventas

El presente documento describe la arquitectura de software propuesta para el Sistema de Inventario y Ventas orientado a pequeñas y medianas empresas (PYMEs). La solución implementa una arquitectura de tres capas con enfoque modular, permitiendo escalabilidad, mantenibilidad y separación clara de responsabilidades.

1. Visión General de la Arquitectura

La arquitectura del sistema se encuentra dividida en tres capas principales: la capa de presentación, la capa de lógica de negocio y la capa de datos. Cada una posee responsabilidades específicas dentro del funcionamiento del software.

Arquitectura del Sistema de Inventario y Ventas

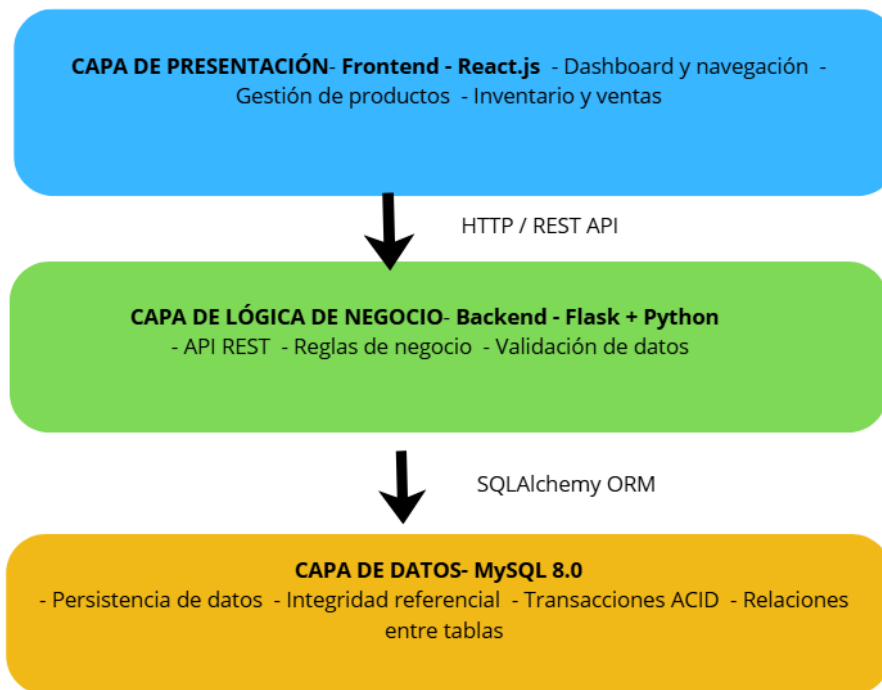


Figura 1. Arquitectura general del sistema.

1.1 Justificación de la Arquitectura

- Separación clara de responsabilidades entre frontend, backend y base de datos.
- Facilidad de mantenimiento y actualización de módulos.
- Escalabilidad para crecimiento de usuarios y volumen de datos.
- Mayor seguridad y control sobre las operaciones del sistema.
- Compatibilidad con futuras integraciones tecnológicas.

2. Capa de Presentación – Frontend

La capa de presentación será desarrollada utilizando React.js, permitiendo interfaces dinámicas, reutilización de componentes y una experiencia de usuario fluida e interactiva.

- Dashboard principal con métricas de ventas y alertas de stock.
- Gestión de productos mediante operaciones CRUD.
- Módulo de inventario y control de stock.
- Registro de ventas y generación de comprobantes.
- Visualización de reportes y gráficos estadísticos.
- Sistema de autenticación y gestión de usuarios.

3. Capa de Lógica de Negocio – Backend

El backend será desarrollado utilizando Python y Flask, encargándose de procesar las reglas de negocio, autenticación, validaciones y manejo de transacciones críticas.

- API REST para comunicación con el frontend.
- Autenticación basada en JWT.
- Validación de datos mediante Marshmallow.
- Gestión modular de productos, ventas y clientes.
- Procesamiento de reportes y alertas de stock mínimo.
- Control de errores y manejo de excepciones.

4. Capa de Datos – MySQL

La persistencia de información será administrada mediante MySQL 8.0, garantizando integridad de datos, relaciones estructuradas y soporte para múltiples usuarios concurrentes.

- USUARIO
- CLIENTE
- PROVEEDOR
- PRODUCTO
- VENTA
- COMPRA
- DETALLE_VENTA
- DETALLE_COMPRA
- CATEGORIA

5. Relación con los Requerimientos del Sistema

La arquitectura planteada responde directamente a los requerimientos funcionales y no funcionales definidos para el proyecto.

Requerimiento	Implementación Arquitectónica
Gestión de usuarios y roles	JWT y middleware de autorización.
Registro de ventas	Módulo de ventas con transacciones ACID.
Control de inventario	Actualización automática de stock.
Generación de reportes	Frontend React con gráficos y estadísticas.
Seguridad	bcrypt, JWT y ORM SQLAlchemy.
Escalabilidad	Arquitectura modular desacoplada.

6. Seguridad del Sistema

- Autenticación mediante JSON Web Tokens (JWT).
- Contraseñas almacenadas con hash bcrypt.
- Protección contra inyección SQL usando SQLAlchemy.
- Control de acceso basado en roles.
- Uso de variables de entorno para credenciales sensibles.
- Validación y sanitización de datos de entrada.

7. Flujo Crítico: Registro de Venta

1. El vendedor inicia sesión en el sistema.
2. Selecciona productos y cantidades.
3. El frontend envía la solicitud mediante la API REST.
4. El backend valida stock y procesa la transacción.
5. Se registra la venta y se actualiza el inventario.
6. El sistema genera el comprobante correspondiente.
7. Se retorna la respuesta al usuario.

8. Conclusiones

La arquitectura propuesta proporciona una base sólida para el desarrollo del Sistema de Inventario y Ventas. El uso de React.js, Python Flask y MySQL permite construir una solución moderna, segura, modular y escalable, alineada con los requerimientos funcionales y no funcionales del proyecto.